Contract Playbook Comparison System

Project Proposal for UCS503

Submitted to: Dr. Jeelani Asif

Thapar Institute of Engineering and Technology

Mannat, DSAI* Upasna Wadhwa, DSAI† Yuvicka, DSAI‡

August 17, 2026

1 Higher-order goal

This project focuses on improving the reliability and consistency of corporate legal and procurement decision-making by minimising the manual effort required to review vendor contracts against internal risk policy. While the underlying technique (retrieval-augmented generation over legal text) is broadly applicable, the immediate engineering objective is to translate grounded, citation-backed clause comparison into a reliable, deployable software pipeline.

2 Time-to-value

- We will deliver meaningful updates quickly by prototyping the core retrieve-compare-report workflow on a small playbook (5 to 8 clause types) and validating it against some contracts within a short iteration window.
- We will prioritize fast feedback over perfection by building each pipeline stage (ingestion, retrieval, comparison, reporting) as an independently testable module, so partial progress may be visible every week.
- Success shall be defined by what can be delivered and measured in the first iteration that could be a working retrieval step evaluated against ground-truth clause spans and then improved based on analysis of retrieval and classification errors.

3 Problem Statement

Companies bargain and sign large volumes of vendor, SaaS, and service contracts. Legal and procurement teams maintain an internal *playbook*: a set of pre-approved clause positions representing acceptable risk (e.g., minimum liability caps, maximum termination notice periods, and compulsory auto-renewal opt-outs). When a new contract arrives, a reviewer must physically read the document (often 20–100+ pages), locate each relevant clause, compare it against the playbook, and flag variations. Common pain points include:

- **Slow, non-scaling review:** manual review takes much time per contract and does not scale with a large number of contracts.
- **Inconsistent outcomes:** different reviewers catch different variations, so review quality varies by who reviews it.

*Roll No: 1024240023, Email: mmannat_be24@thapar.edu
†Roll No: 1024240022, Email: uwadhwa_be24@thapar.edu
‡Roll No: 1024240016, Email: yyuvicka_be24@thapar.edu

- **No audit trail:** verdicts are hardly linked back to the exact source text that produced them, making them hard to prove or dispute later.
- **Hallucination risk of naive LLM use:** feeding an entire contract to a general-purpose LLM and asking, "Does this match our policy?" produces unverifiable, sometimes fabricated, answers unacceptable when a missed liability clause has real financial consequences.

Why this matters now: Contract Lifecycle Management (CLM) is an established, funded software category (e.g., Ironclad, LinkSquares, Lawgeex, Evisort), confirming this is a real workflow gap and not a contrived exercise. A retrieval-grounded approach is well suited here precisely because the cost of an ungrounded answer is high.

4 Proposed Solution

4.1 Overview

Building a retrieval-augmented pipeline, the *Contract Playbook Comparison System*, with the following parts:

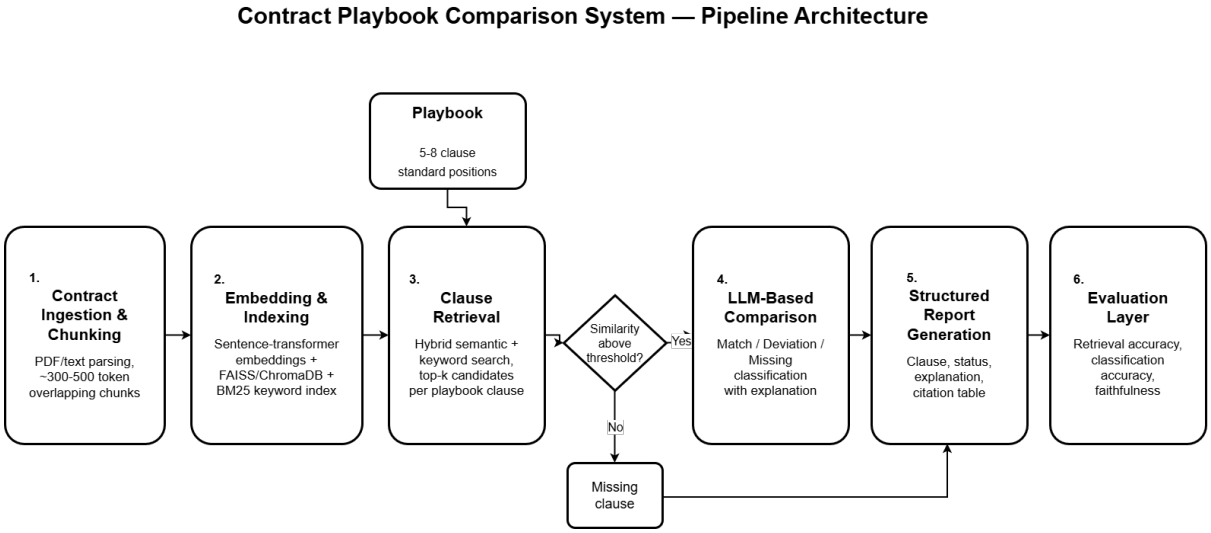

Figure 1: End-to-end pipeline architecture: contract ingestion, embedding/indexing, hybrid clause retrieval, LLM-based comparison, structured reporting, and evaluation.

- **Playbook definition:** 5 to 8 clause categories (e.g., Termination, Liability Cap, Auto-Renewal, Indemnification, Governing Law) each with a standard documented position.
- **Clause retrieval:** hybrid semantic + keyword search that locates the section(s) of an incoming contract mapping it to each playbook clause.
- **Clause comparison:** an LLM classifies each retrieved section as *Match*, *Deviation*, or *Missing* according to the playbook position, with a natural language explanation.
- **Structured report:** one row per playbook clause, showing status, explanation, and an exact citation into the source contract.

4.2 Core workflow

1. An incoming contract is parsed and chunked into indexable segments.
2. For each playbook clause type, the retrieval stage will return the top-k candidate contract chunks.
3. If no chunk passes a similarity threshold, the clause is classified *Missing* directly and hence the LLM is never asked to guess from nothing.
4. Otherwise, the LLM compares the retrieved text against the playbook position and returns *Match* / *Deviation* with a one-sentence explanation.
5. Results are put into a structured report with source citations.

4.3 Operational constraints

- To use a well-established public dataset (CUAD) rather than sourcing and labelling proprietary contracts.
- Avoid dependence on unproven algorithms rather: favour standard, well-documented retrieval and evaluation techniques.
- Designing for a reproducible local/demo deployment (Streamlit or Gradio) rather than production-grade infrastructure.

5 Solution Approach

This is an engineering course project, so we focus on building and validating a modular pipeline rather than on unusual algorithm research.

5.1 Dataset and grounding

We will use CUAD (Contract Understanding Atticus Dataset): 510 real commercial contracts with 13,000+ expert-annotated clause spans across 41 categories.

- Select 5–8 clause categories with reasonable positive-example counts in CUAD and define a documented standard position for each.
- Use held-out CUAD contracts as test inputs, giving automatic ground truth for retrieval evaluation (does the system find the correct labelled span?).
- Hand-label a small set (˜50–100) of clause-comparison pairs as Match/Variation/Missing for categorising evaluation.

5.2 Pipeline architecture

As illustrated in Figure 1, this is a staged retrieval-augmented pipeline:

- **Ingestion:** PDF/text parsing (pdfplumber/PyMuPDF) and intersecting chunking (˜300–500 tokens, ˜15% overlap) tuned for long legal clauses.
- **Indexing:** sentence-transformer embeddings stored in FAISS/ChromaDB, alongside a BM25 keyword index for hybrid retrieval, important because exact legal terminology ("indemnify", "force majeure") often matters more than pure semantic similarity.

- **Comparison:** a structured LLM prompt that receives only the playbook position and the retrieved chunk(s), prohibiting the model from answering off of unretrieved, hallucinated context.
- **Reporting:** a single table with clause type, status, explanation, and citation as the reviewer-facing output.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically run retrieval and classification validation scripts on every change to the pipeline.
- Produce a repeatable demo build (Streamlit/Gradio) for weekly review.
- Enable safe, frequent iteration on every pipeline stage without reevaluating the whole system manually.

6 Evaluation Criterion

We define success using metrics bind to each pipeline stage, each with its own ground truth.

6.1 Primary evaluation metric

Retrieval accuracy: Does the system return the correct contract span for each playbook clause type, measured directly against CUAD's labelled spans?

- Target: Recall@k (k=3-5) and mean reciprocal rank (MRR) reported per clause category.
- Attribution: computed automatically by matching retrieved chunk offsets against CUAD ground-truth spans.

6.2 Secondary evaluation metrics

- **Classification accuracy:** agreement between the pipeline's match/deviation/missing verdicts and the team's manually-labelled test set.
- **Faithfulness:** Whether the LLM's explanation is led by the retrieved text (checked manually on a selection and optionally via an NLI-based check as in the RAGAS framework).
- **Coverage:** percentage of playbook clause types for which the system returns a confident retrieval (above threshold) versus falling back to *Missing*.

6.3 Validation plan

- Evaluating retrieval and classification on a held-out split of CUAD contracts not used to design the playbook positions.
- Comparing hybrid (semantic + BM25) retrieval against semantic-only retrieval as an ablation, to justify the hybrid design choice.

7 Scalability

The system should scale in both conceptual and practical senses.

1. **Scalable (conceptually):** the pipeline should support supplementary playbook clause types and larger contract volumes by
    - decoupling ingestion, retrieval, comparison, and journalising stages,
    - indexing embeddings for sub-linear nearest-neighbour lookup,
    - batching LLM comparison calls independently per clause type.
2. **Operationally feasible (practically):** the demo should run on commodity resources:
    - a local or lightly-hosted vector index (FAISS/ChromaDB),
    - any accessible LLM API or a small open model to control cost,
    - CI-run evaluation scripts that do not require GPU infrastructure.

8 Engine availability heuristic

A mature set of "engines" (tooling/libraries) is available to implement this without inventing a new infrastructure:

- Sentence-embedding models (`sentence-transformers`, e.g. `all-mpnet-base-v2`, or a legal-domain model such as Legal-BERT).
- A vector store (FAISS or ChromaDB) and a keyword index (`rank_bm25`).
- An accessible LLM API for the comparison/classification step.
- An evaluation framework purpose-built for RAG (RAGAS) instead of a manual evaluation system.
- A lightweight demo UI framework (Streamlit or Gradio).

We will use these existing components to focus on pipeline correctness and evaluation quality rather than infrastructure-building.

9 Project scope and deliverables

9.1 Initial deliverable

- Playbook definition for 5–8 clause categories with documented standard positions.
- Ingestion and chunking pipeline for CUAD contract text.
- Embedding index (FAISS/ChromaDB) with basic semantic retrieval.
- Retrieval evaluation against CUAD ground-truth spans (Recall@k, MRR).
- CI: automated evaluation run + linting on every change.

9.2 Subsequent deliverables

- Hybrid retrieval (BM25 + semantic) and ablation comparison against semantic-only retrieval.
- LLM-based clause comparison with structured Match/Deviation/Missing output.
- Hand-labelled classification test set and accuracy/faithfulness evaluation.
- Streamlit/Gradio demo: contract selection → playbook comparison → cited report.

10 Risks and mitigations

- **LLM hallucination even with retrieved context:** keep comparison prompts strict ("only use the provided text; say 'cannot determine' if unclear"), and always surface the retrieved source text alongside the verdict so it is independently verifiable.
- **Class imbalance in CUAD categories:** check per-category label counts in week 1 and choose playbook clause types with enough representation.
- **Fuzzy clause boundaries:** allow retrieval to return multiple adjacent chunks rather than a single best match, since clauses can span paragraphs.
- **Domain-specific phrasing confusing embeddings:** mitigate with hybrid (semantic + keyword) retrieval, since exact legal terms often carry more signal than semantic similarity alone.

11 Summary

This project suggests a retrieval-augmented pipeline that automates contract clause review against an internal playbook, producing grounded, citation-backed verdicts instead of unverifiable LLM output. It meets the course criteria by decomposing the system into independently testable modules, defining measurable validation criteria at each stage (retrieval accuracy, classification accuracy, and faithfulness), and grounding the whole design in a well-established public legal-NLP benchmark (CUAD) rather than research-driven novelty.